CS6868 Research Mini-Project

Concurrent Hash Maps: Striped Locking vs. Split-Ordered Lists

Ogirala Srihasa
cs25m034@smail.iitm.ac.in

Botla Venkata Ramamohan Rao
cs25m017@smail.iitm.ac.in

May 1, 2026

Abstract

Concurrent hash maps are a foundational data structure in multicore software, requiring careful trade-offs between implementation complexity, contention overhead, and scalability. We implement and comparatively evaluate two designs in OCaml 5: a *striped-locking* hash map, which partitions buckets across mutex-protected stripes with stop-the-world resizing, and a *lock-free split-ordered list* hash map, which encodes all entries in a single CAS-based linked list sorted by bit-reversed hash keys, enabling incremental resizing without rehashing. Both implementations are realised as functors over arbitrary key types and verified for correctness using QCheck-Lin, QCheck-STM (sequential consistency), and Thread Sanitizer (data-race detection), which together uncovered and guided the resolution of bugs during development. Benchmarks across 1–8 threads and three workload profiles — read-heavy, write-heavy, and mixed — show that the split-ordered design achieves up to 4.2× higher throughput under read-heavy workloads at 8 threads and maintains 3.5× higher throughput during active resizing, at the cost of 1.7× higher memory overhead. These results confirm that lock-free designs offer meaningful scalability advantages in OCaml 5's multicore runtime.

1 Goals

Hash maps are among the most widely used data structures in systems programming, and making them thread-safe without sacrificing throughput is a central challenge in concurrent programming. This project implements and compares two fundamentally different approaches to concurrent hashing in OCaml 5:

1. **Striped locking** — the bucket array is partitioned into mutex-protected stripes, enabling coarse-grained parallelism. This design extends the striped hash set presented in Chapter 13 of *The Art of Multiprocessor Programming* [1] to a full key-value hash map, using standard `Mutex` primitives for stripe-level locking with dynamic resizing.

2. **Split-ordered lists** — a lock-free design introduced by Shalev and Shavit [2] and discussed in Chapter 13 of *The Art of Multiprocessor Programming* [1]. We extend the original hash set formulation to support arbitrary key-value mappings, enabling lock-free resizing without rehashing.

Both implementations support `insert`, `find`, `remove`, and `size`, with dynamic resizing under load.

Research question. *Does the lock-free split-ordered list design provide meaningful throughput advantages over striped locking under read-heavy workloads, and how do the two approaches compare during concurrent resizing?*

2 Background

2.1 Striped locking

A striped hash map maintains an array of *buckets*, where each bucket is a list of key-value pairs sharing the same hash modulo the bucket count. To enable concurrency without a single global lock, the array is partitioned into N *stripes*, where bucket i is guarded by stripe $i \bmod N$ [1]. Operations on different stripes proceed in parallel; those on the same stripe serialise. Each operation computes the target bucket, acquires the corresponding stripe lock, and scans the bucket's list for the key.

As nodes accumulate, bucket lists grow and lookup degrades from $O(1)$ toward $O(n)$. When the load factor — stored entries divided by bucket count — exceeds 0.75, the map *resizes*: all N stripe locks are acquired in ascending index order (to prevent deadlock), the bucket array is doubled, every entry is rehashed into its new position, and the locks are released. No other thread may access the map during this window, making resize a *stop-the-world* event and the primary scalability bottleneck of the striped design.

2.2 Split-ordered lists

A split-ordered hash map stores all nodes in a *single* lock-free linked list sorted by a *split-ordered key*: the bit-reversal of the node's hash, with the LSB forced to 1 for data nodes and to 0 for sentinel nodes. The LSB convention ensures every sentinel sorts strictly before the data nodes of its bucket. Ties in split-ordered key (caused by hash collisions) are broken by `K.compare` on the original keys.

To avoid scanning from the list head, the map maintains an array of *sentinel* pointers as indexed entry points at bucket boundaries. An operation on bucket b begins traversal from b's sentinel, bounding the scan to that bucket's nodes alone. All insertions and removals use compare-and-set (CAS) on node next-pointers, following the Harris two-phase protocol from Lecture 7.

Resizing is *incremental*: when the load factor exceeds 2.0, the logical bucket count is doubled from B to $2B$ with a single CAS on an atomic counter. No node moves. Because bit-reversal maps higher hash bits to lower sort-key positions, the nodes of bucket b already partition in sorted order into those that remain in b and those that migrate to $b + B$ — the split requires no relinking.

Sentinel nodes are created *lazily* on first access. The sentinel for bucket b is inserted starting from the sentinel of b's *parent* — the bucket obtained by stripping the highest set bit of b (e.g. $\text{parent}(5) = \text{parent}(101_2) = 001_2 = 1$) — which is always the closest preceding entry point in the sorted list. If the parent sentinel does not yet exist it is initialised recursively; the recursion terminates at bucket 0, whose sentinel is the permanent list head. Because resize acquires no locks and touches no existing nodes, all threads continue operating throughout — making resize fully concurrent and the primary scalability advantage over striped locking.

3 Tasks Undertaken

3.1 Implementation

The complete implementation can be found at the public github repo

Both hash maps are realised as OCaml functors parameterised over a `KEY` module (defined in `hashmap_intf.ml`), which must supply a type `t`, a `hash` function, an `equal` predicate, and a `compare` function. Both functors produce a module satisfying the `HASHMAP` signature, exposing `create`, `insert`, `find`, `remove`, and `size`. This common interface allows the benchmark and

test harnesses to swap one implementation for the other with a single module alias change. For the lock free implementation we used the `AtomicMarkableRef` from lecture 7

3.1.1 Striped locking hash map (`striped_hashmap.ml`)

Data structure. The striped hash map stores nodes in a bucket array, where each bucket is an OCaml association list of `(key, value)` pairs sharing the same hash modulo the bucket count (separate chaining). The record type is:

```
type 'v t = {
  buckets  : (K.t * 'v) list array Atomic.t;
  locks    : Mutex.t array;
  num_stripes      : int;
  size     : int Atomic.t;
  capacity : int Atomic.t;
  load_factor_threshold : float;
  resize_lock : Mutex.t;
}
```

Both `buckets` and `capacity` are wrapped in `Atomic.t`. This was a deliberate design decision made after ThreadSanitizer (Section 3.2.4) reported data races on both fields: `capacity` was read by `should_resize` without any lock held, and the `buckets` pointer was swapped by `resize` while readers held only one stripe lock. Making both fields atomic eliminates the races without requiring callers to hold any additional locks.

Stripe assignment. Bucket i is guarded by stripe $i \bmod N$, where N is the configurable stripe count (default 16). A `Mutex.t array` of length N is allocated at creation and never changes. The number of stripes is kept less than or equal to the initial capacity to ensure every stripe guards at least one bucket.

Operations. Each of `insert`, `find`, and `remove` follows the same retry protocol:

1. Compute the bucket index b and stripe index $s = b \bmod N$.
2. Acquire `locks.(s)`.
3. Recompute the bucket index b' under the lock.
4. If $b \neq b'$, a concurrent resize changed `capacity` between steps 1 and 2, moving the key to a different stripe. Release the lock and restart from step 1.
5. Otherwise, operate on the chain at `buckets.(b)` and release.

The re-validation in step 4 is a subtle but essential correctness condition. Without it, a thread that computed its bucket index before a resize could write to the wrong bucket after the resize doubled the array.

`insert` performs an upsert: if the key already exists in the chain, it rebuilds the chain with the new value using `List.map` rather than removing and re-inserting, avoiding a window in which the key is temporarily absent. If the key is new, it is prepended to the chain and `size` is incremented atomically.

Dynamic resizing. After every `insert`, `should_resize` checks whether the load factor |size|/|capacity| exceeds the threshold (default 0.75). If so, `resize` is called outside the stripe lock. `resize` first acquires a dedicated `resize_lock` (so only one resize runs at a time), double-checks the condition, then acquires all N stripe locks in strictly ascending index order. Ascending order prevents deadlock: any two threads that attempt to acquire the same set of locks will always request them in the same order. With all stripes held, the bucket array is doubled, every entry is rehashed into the new array, and `buckets` and `capacity` are updated atomically via `Atomic.set` before releasing the locks. This is a stop-the-world event: no other thread may read or write the map during the resize window.

3.1.2 Split-ordered list hash map (`split_ordered_hashmap.ml`)

Data structure. The split-ordered map stores all nodes in a single lock-free linked list sorted by a *split-ordered key*. Each node in the list has the following type:

```
type 'v node = {
  so_key   : int;
  real_key : K.t;
  value    : 'v option Atomic.t;
  next     : 'v node AMR.t;
}
```

`so_key` is the bit-reversed sort key that determines the node's position in the global list. `real_key` is the original unhashed key, used as a tiebreaker when two nodes share the same `so_key` due to a hash collision. `value` is wrapped in `Atomic.t` to allow in-place value updates without removing and re-inserting the node, eliminating any window in which the key would be temporarily absent. `next` is an `AtomicMarkableRef` rather than a plain pointer, pairing the next-node reference with a boolean mark bit; setting the mark bit to `true` signifies that this node is logically deleted and should be physically unlinked during the next traversal of that region of the list. Sentinel nodes use the same type but carry a placeholder `sentinel_dummy_key` (an `Obj.magic 0` value) in `real_key` and `None` in `value`; the `node_less_than` and `node_equal_key` helpers guard against passing this placeholder to `K.compare` by only invoking it when both nodes are regular (LSB = 1 in `so_key`).

The table itself is represented by the following record:

```
type 'v t = {
  head         : 'v node;
  buckets      : 'v node option Atomic.t array;
  bucket_count : int Atomic.t;
  max_buckets  : int;
  size         : int Atomic.t;
  load_factor_threshold : float;
}
```

`max_buckets` is a fixed physical upper bound allocated at creation (default: initial capacity $\times$ 256, permitting up to eight doublings). `bucket_count` is the logical active size and grows via CAS during resize. `buckets` is a pre-allocated sentinel pointer array; slots beyond `bucket_count` are uninitialised (`None`) until lazily populated.

Key encoding and bit reversal. Two functions encode keys as sort keys for the global list:

```
let make_regular_key h  = (reverse_bits h) lor 1
let make_sentinel_key i = (reverse_bits i) land (lnot 1)
```

`reverse_bits` reverses all 62 non-sign bits of an OCaml integer (63 bits on 64-bit systems, minus the sign bit). Forcing the LSB to 1 for regular nodes and to 0 for sentinel nodes ensures every sentinel sorts strictly before the data nodes of its own bucket, since they share the same upper bits but differ in the LSB position ($0 < 1$).

The raw hash is masked to `max_int - 1` before bit reversal, ensuring the result never equals `max_int`, which is reserved for the tail sentinel's sort key.

Collision handling. Because `hash_key` can map two distinct keys to the same integer, two regular nodes may share the same split-ordered key. To prevent silent key loss or incorrect value returns, all list operations use a *composite* sort key `(so_key, real_key)`: nodes with equal `so_key` are further ordered by `K.compare` on their original keys. The helpers `node_less_than` and `node_equal_key` only invoke `K.compare` when both nodes are regular (LSB = 1), so the `sentinel_dummy_key` placeholder (an `Obj.magic 0` value used for sentinel and tail nodes) is never passed to `K.compare`.

Lock-free list operations. `locate` finds the `(pred, curr)` window around a target key, physically snipping out logically deleted nodes along the way via CAS. If a snip CAS fails (a concurrent thread already snipped the node), `locate` restarts from the sentinel rather than the list head, bounding the retry scope.

Removal uses the Harris two-phase protocol: `list_remove` first marks `curr.next` atomically (logical deletion), then attempts a best-effort CAS to physically unlink `curr` from `pred.next`. Physical unlinking is best-effort because subsequent `locate` calls will snip any remaining marked nodes during their traversal. The logical mark is critical: without it, a concurrent `list_insert` immediately after `curr` could be silently lost when the physical-unlink CAS discards the freshly inserted node.

`insert` performs an atomic in-place value update when a key already exists: rather than removing and re-inserting (which would create a window where the key is absent), it locates the existing node and calls `Atomic.set` on its `value` field directly.

Lazy sentinel initialisation. `get_bucket t idx` returns the sentinel node for bucket `idx`, calling `init_bucket t idx` if the slot is `None`. `init_bucket` inserts the new sentinel starting from the *parent* bucket's sentinel rather than the list head. The parent of bucket b is obtained by stripping the highest set bit of b: for example, $\text{parent}(5) = \text{parent}(101_2) = 001_2 = 1$. This relationship reflects the doubling structure of resize: bucket b is created by splitting its parent, so the parent sentinel is always the closest preceding entry point in the sorted list. If the parent sentinel does not yet exist, `init_bucket` initialises it recursively first; the recursion always terminates at bucket 0, whose sentinel is the permanent list head. Because multiple threads may race to initialise the same sentinel, `init_bucket` always calls `list_find` after the insert attempt to read back whichever sentinel won the race, and stores that pointer into the `buckets` array.

Incremental lock-free resizing. `maybe_resize` doubles `bucket_count` with a single CAS if the load factor exceeds the threshold (default 2.0, higher than striped because per-bucket traversal cost is lower). No node moves, no lock is acquired, and no thread is blocked. When the bucket count doubles from B to $2B$, the nodes in bucket b naturally split into those that remain in bucket b and those that now belong to bucket $b + B$, as determined by the next bit of their bit-reversed hash. This split is already encoded in the existing sort order of the list and requires no relinking.

3.2 Testing and Verification

We applied three complementary verification techniques to both implementations: manual unit tests, property-based concurrency tests (QCheck-Lin and QCheck-STM), and ThreadSanitizer. Together they uncovered and guided the resolution of bugs during development.

3.2.1 Manual unit tests

Both `test_striped.ml` and `test_split_ordered.ml` have implementation of the following tests: sequential operations, error handling, concurrent inserts, concurrent mixed operations, resize correctness, no-lost-items (striped only), concurrent resize stress (split-ordered only), negative integer keys, string keys, and float keys.

3.2.2 QCheck-STM

`qcheck_stm_striped.ml` and `qcheck_stm_split_ordered.ml` use QCheck-STM to test both implementations against a sequential reference model. The model is OCaml's `Hashtbl` with functional semantics: the state is an association list of `(key, value)` pairs, and each command (`Insert`, `Find`, `Remove`, `Size`) has a corresponding `next_state` transformer and `postcond` check.

3.2.3 QCheck-Lin

`qcheck_lin_striped.ml` and `qcheck_lin_split_ordered.ml` each test generates 1 000 random concurrent `insert/find/remove` sequences across two domains and checks that some valid sequential interleaving of the observed results exists, again using `Hashtbl` as the reference model.

The `size` operation is deliberately excluded from the Lin tests for both implementations. In both designs, the `size` counter (`int Atomic.t`) is incremented or decremented *after* the structural change (insert or remove) has been committed. There is therefore a brief window in which the structure has changed but `size` has not yet been updated, making `size` inherently non-linearizable with respect to the structural operations. This is an accepted and documented limitation of the counter design.

3.2.4 ThreadSanitizer (TSAN)

Both implementations were run under ThreadSanitizer using OCaml 5's dedicated `5.4.0+tsan` opam switch, which instruments the runtime for data-race detection. The manual concurrent tests (`test_striped.ml`, `test_split_ordered.ml`) were used as the TSAN workloads since they exercise the highest level of concurrency.

TSAN reported two distinct data races on the striped implementation and no races on the split-ordered implementation.

Race 1 — `should_resize` reads `capacity` with no lock. `should_resize` was called before acquiring any stripe lock, and read `t.capacity` as a plain `int` while `resize` was writing it under all stripe locks. TSAN identified the conflict as a write in `camlStriped_hashmap.fun_697` (the resize closure) racing with a read in `camlStriped_hashmap.should_resize_394`.

Race 2 — `bucket_for_key` reads `capacity` before acquiring the stripe lock. In the `attempt` loop, `bucket_for_key` is called *first* to compute the initial bucket index, and only then is the corresponding stripe lock acquired. This first call reads `t.capacity` with *no* stripe lock held. Concurrently, `resize` writes `t.capacity` while holding all N stripe locks. The read and write are therefore wholly unsynchronised.

Fix. Both fields `capacity` and `buckets` were changed from plain mutable record fields to `int Atomic.t` and `(K.t * 'v) list array Atomic.t` respectively. All reads were updated to `Atomic.get` and all writes to `Atomic.set`. After this change, TSAN reported no races on either implementation.

3.2.5 Bugs discovered

The three verification layers together uncovered bugs during development:

1. **Sentinel key collision with tail** (split-ordered) — using `land max_int` instead of `land (max_int - 1)` in `hash_key` allowed the tail sentinel's sort key (`max_int`) to be produced by a real key, breaking list termination. Caught by the manual boundary-key tests.

2. **Wrong bucket after resize in striped** — the retry loop in `insert/find/remove` did not re-validate the bucket index after acquiring the stripe lock. A concurrent resize between the index computation and the lock acquisition could place the operation in the wrong bucket. Caught by the STM concurrent tests and fixed by re-computing and comparing the bucket index inside the lock.

3. **Data race on `capacity` and `buckets`** (striped) — as described in Section 3.2.4, both fields were unprotected plain record fields. Caught by TSAN and fixed by making both `Atomic.t`.

4 Evaluation

4.1 Experimental setup

All benchmarks were run on an Apple MacBook Air M4 (8 performance cores, 16 GB RAM) under OCaml 5 with the default runtime. Thread counts ranged from 1 to 8. Each configuration was repeated 10 times and averaged. Three workload profiles were tested: read-heavy (90/5/5 find/insert/remove), write-heavy (10/45/45), and mixed (50/25/25), each with 500,000 operations per thread and 1,000 pre-populated entries. A separate resize benchmark used an 80% insert workload starting from capacity 4 to stress concurrent resizing.

4.2 Throughput scaling

Figures 1 to 3 plots throughput in millions of operations per second as thread count increases across all three workload profiles.

Observations. At 1 thread, the striped map is faster across all workloads due to its lower per-operation cost: bucket-chain reads require no CAS and no atomic pointer traversal. However, stripe-lock contention causes striped throughput to degrade beyond 4 threads, settling at 6–10 Mops/s by 8 threads regardless of workload.

The split-ordered list scales near-linearly through all 8 performance cores, peaking at 42.9 Mops/s under the read-heavy workload at 8 threads — 4.5× the striped map. Under write-heavy load the advantage is reduced (3.2× at 8 threads) because failed CAS retries on the linked list are more expensive than a simple list mutation under a stripe lock. Nevertheless, the lock-free design retains positive scaling through 8 threads across all workloads, whereas the striped map degrades monotonically beyond 4 threads.

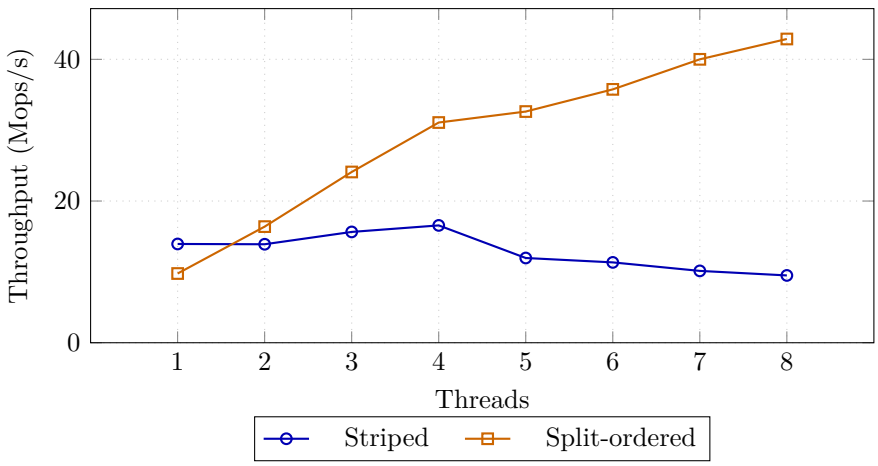

Figure 1: Throughput (Mops/s) vs. thread count — read-heavy workload (90/5/5 find/insert/remove, 500K ops/thread, average of 10 runs).

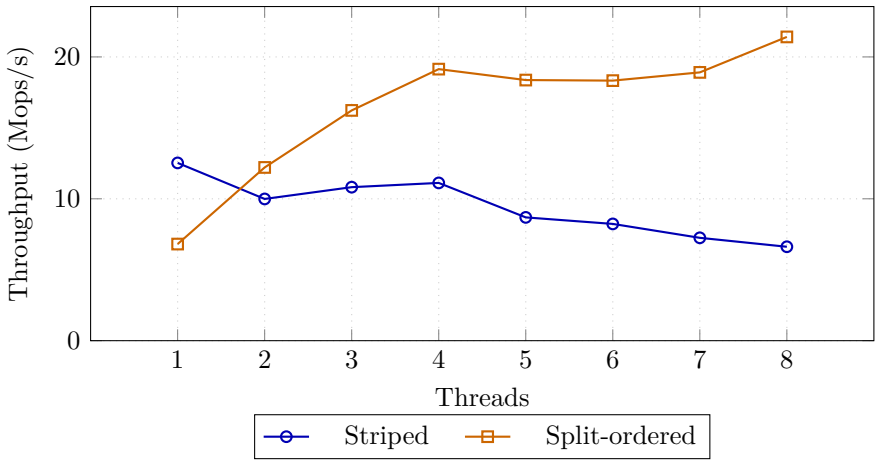

Figure 2: Throughput (Mops/s) vs. thread count — write-heavy workload (10/45/45 find/insert/remove, 500K ops/thread, average of 10 runs).

4.3 Resize benchmark

Observations. The resize benchmark most clearly separates the two designs. The striped map's stop-the-world rehash stalls all threads while one acquires every stripe lock, copies every entry, then releases — producing severe throughput degradation (3.5 Mops/s at 3 threads) that only partially recovers at higher thread counts as the amortised resize cost is spread across more operations.

The split-ordered list's resize is a single CAS on the bucket counter, with no node movement and lazy sentinel insertion. This allows the map to sustain 13–20 Mops/s through 8 threads, peaking at 3 threads (5.6× the striped map at that point).

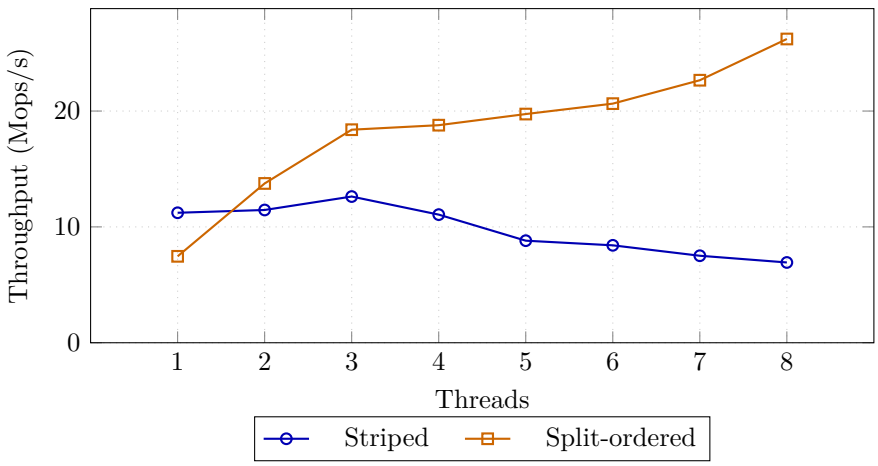

Figure 3: Throughput (Mops/s) vs. thread count — mixed workload (50/25/25 find/insert/remove, 500K ops/thread, average of 10 runs).

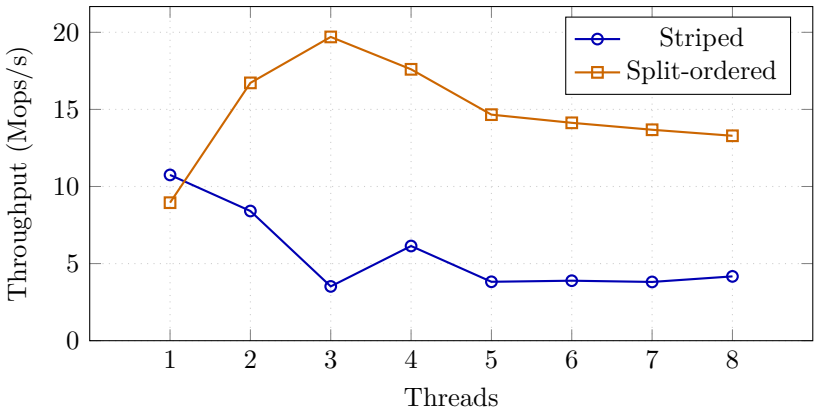

Figure 4: Throughput during active resizing (80% inserts, initial capacity 4, average of 10 runs, 200K ops/thread).

4.4 Memory overhead

N	Striped (words)	(KB)	Split (words)	(KB)	Ratio
100	965	7	14,074	109	14.6×
1,000	8,157	63	29,712	232	3.6×
10,000	76,493	597	180,324	1,408	2.4×
100,000	862,253	6,736	1,441,010	11,257	1.7×
500,000	4,048,685	31,630	7,041,010	55,007	1.7×

Table 1: Live-heap memory at different map sizes, measured via `Gc.compact` before and after N insertions.

Observations. At small scale ($N = 100$) the split-ordered list consumes 14.6× more memory, owing to its pre-allocated sentinel array and its four-word node layout (`so_key`, `real_key`, `value Atomic.t`, `next AtomicMarkableRef.t`) versus the striped map's two-word `(key, value)` cons cell. As N grows the ratio converges to ∼1.7× at 100,000 entries and beyond — a modest premium given the throughput advantages at scale.

5 Reflection on the Use of LLMs

Tools and models used. We used *Claude Opus 4.6* (via the Anthropic Claude desktop application with the Planner/computer-use interface) throughout the project. The model was used for: generating the initial project scaffold and both hash map implementations, writing all test files (manual, QCheck-Lin, QCheck-STM), writing the benchmark harness, debugging data races and correctness failures.We used *Gemini 3.1 pro, Claude Sonnet 4.6* for concept explanations, ppt generation.

What worked well. The LLM was highly productive for generating boilerplate and scaffolding. The initial project structure — `lib/`, `test/`, `bench/`, `dune` files, `Makefile` — was produced correctly in a single pass with no edits needed. The functor refactor (converting both implementations from hardcoded `int` keys to `Make(K : KEY)`) was executed across twelve files simultaneously and compiled cleanly on the first build. When TSAN reported two data races on `capacity` and `buckets`, the model correctly identified that plain mutable fields read outside locks were the cause and changed them to `Atomic.t` in one edit.

What did not work. The most significant failure was the initial treatment of negative integer keys. The model first introduced `(Hashtbl.hash k) land (max_int - 1)` as a safe hash function, then separately failed to realise that `Hashtbl.hash` is a 30-bit hash that can produce collisions between distinct keys. When we pointed out that two different keys could map to the same `so_key`, the model acknowledged the bug, added `real_key` comparisons and secondary sorting by `K.compare` throughout the list operations. The model also initially used `nat_small` (non-negative integers only) in both QCheck-Lin tests, which meant the negative-key fix was never actually exercised by the randomised tests — a coverage gap that had to be pointed out explicitly before the generator was corrected to `int`.

What was surprising. We were surprised by how accurately the model could read and interpret TSAN output. Given a multi-hundred-line TSAN report, it correctly extracted the two racing stack frames (`camlStriped_hashmap.fun_697` writing `t.buckets` and `camlStriped_hashmap.should_resize_394` reading it outside any lock) and produced the exact fix without further prompting.

What was difficult. The model was a poor substitute for genuine understanding of the core algorithmic concepts. Asking it to explain *why* bit-reversal preserves the bucket-splitting invariant, or why forcing $\mathrm{LSB} = 0$ on sentinel keys and $\mathrm{LSB} = 1$ on regular keys produces the correct ordering, produced fluent prose that restated the property without building any intuition for it. We had to work through concrete bit-arithmetic examples by hand and read the AoMPP chapter 13 [1] directly before these invariants became clear. Wherever the project required understanding *why* a concurrent invariant holds rather than *what* the code should look like, the model's output needed independent verification against primary sources.

Your overall assessment. The LLM substantially accelerated implementation and eliminated nearly all syntactic and boilerplate work, but it required careful human oversight at every correctness boundary. We would use it the same way again, but with earlier and more systematic adversarial review of any code touching concurrency invariants.

6 Conclusion

Research question. Yes, the lock-free split-ordered list provides meaningful throughput advantages over striped locking under read-heavy workloads. At 8 threads the split-ordered

map achieves 42.9 Mops/s against the striped map's 9.5 Mops/s — a 4.5× advantage — and scales near-linearly through all 8 performance cores while striped throughput degrades beyond 4 threads due to stripe-lock contention. The advantage persists across all three workload profiles, though it is reduced under write-heavy load (3.2× at 8 threads) where CAS retries on the linked list are more expensive than a guarded list mutation. During concurrent resizing the split-ordered design is decisively better: its single-CAS incremental resize sustains 13–20 Mops/s while the striped map's stop-the-world rehash drops to 3.5 Mops/s at 3 threads, recovering only partially at higher thread counts.

Limitations. Several limitations should be acknowledged. First, all benchmarks were run on a single machine (Apple M4); results may differ on x86 hardware with different cache coherence and memory ordering characteristics. Second, the split-ordered map pre-allocates a fixed-size sentinel array, imposing a hard upper bound on the number of doublings and a 14.6× memory overhead at small scale. Finally, the striped implementation still has a theoretical starvation risk during resize, as incoming inserts could repeatedly win stripe locks before the resize thread acquires them all, though this was not observed in practice.

Future work. Given more time we would pursue three directions. First, making the sentinel array itself dynamically growable would remove the pre-allocation upper bound and reduce small-scale memory overhead. Second, replacing the per-stripe `Mutex` with a reader-writer lock in the striped design would allow concurrent reads under the same stripe, likely recovering some of the throughput gap under read-heavy workloads. Finally, benchmarking under a wider range of hardware (NUMA machines, x86 many-core) and workload distributions would give a more complete picture of where each design excels.

Contributions

Member	Contribution (%)	Main responsibilities
Ogirala Srihasa	50%	Split-ordered implementation, benchmarks, ppt
Ramamohan	50%	Striped implementation,functor refactoring, QCheck tests, report
Total	**100%**	

References

[1] Maurice Herlihy, Nir Shavit, Victor Luchangco, and Michael Spear. *The Art of Multiprocessor Programming.* Morgan Kaufmann, 2nd edition, 2020.

[2] Ori Shalev and Nir Shavit. Split-ordered lists: Lock-free extensible hash tables. *Journal of the ACM*, 53(3):379–405, 2006.